

---

---

TARGET DRIVEN GUIDANCE  
FOR  
EXTREME WEATHER STORYLINES

---

---

MASTER'S THESIS

AUTHOR

**ALAIN JOSS**

ALAIN.JOSS@UZH.CH

SUPERVISORS

MAXIM SAMARIN

SHIRIN GOSHTASBPOUR

PROF. DR. GUILLAUME OBOZINSKI

SWISS DATA SCIENCE CENTER

DATE OF SUBMISSION: AUGUST 15, 2026

# Abstract

# Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                | <b>1</b>  |
| <b>2</b> | <b>Background</b>                                  | <b>4</b>  |
| 2.1      | Flow matching . . . . .                            | 4         |
| 2.1.1    | Conditional probability paths . . . . .            | 5         |
| 2.1.2    | Conditional and marginal velocity fields . . . . . | 6         |
| 2.1.3    | Flow-matching objective . . . . .                  | 6         |
| 2.1.4    | Linear probability path . . . . .                  | 7         |
| 2.2      | Guidance . . . . .                                 | 8         |
| 2.2.1    | Guidance as posterior sampling . . . . .           | 8         |
| 2.2.2    | From scores to flow-matching velocities . . . . .  | 10        |
| 2.2.3    | Guidance-strength schedules . . . . .              | 11        |
| 2.2.4    | Guidance as control . . . . .                      | 12        |
| 2.3      | Guided generation of weather scenarios . . . . .   | 14        |
| 2.4      | ArchesWeather . . . . .                            | 16        |
| 2.4.1    | Dataset . . . . .                                  | 17        |
| 2.4.2    | Forecasting setup . . . . .                        | 17        |
| 2.4.3    | Residual flow model . . . . .                      | 18        |
| 2.4.4    | Sampling . . . . .                                 | 19        |
| <b>3</b> | <b>Specifying events of interest</b>               | <b>21</b> |
| 3.1      | Target variable and region . . . . .               | 22        |
| 3.2      | Target trajectory . . . . .                        | 24        |
| 3.2.1    | Guiding an ensemble . . . . .                      | 24        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 3.2.2    | Prescribing the relative change . . . . .           | 25        |
| 3.3      | Pipeline overview . . . . .                         | 26        |
| <b>4</b> | <b>Guidance design</b>                              | <b>28</b> |
| 4.1      | Target-driven guidance . . . . .                    | 29        |
| 4.1.1    | Guidance objective . . . . .                        | 29        |
| 4.1.2    | Gradient-based flow guidance . . . . .              | 30        |
| 4.2      | Guidance-strength design . . . . .                  | 31        |
| 4.2.1    | Alternative target-realization strategies . . . . . | 32        |
| 4.2.2    | Final flow step . . . . .                           | 32        |
| 4.3      | Target realization by gain adaptation . . . . .     | 33        |
| 4.3.1    | Secant search . . . . .                             | 34        |
| <b>5</b> | <b>Evaluation</b>                                   | <b>36</b> |
| 5.1      | Quantitative evaluation . . . . .                   | 37        |
| 5.1.1    | Q2 . . . . .                                        | 37        |
| 5.1.2    | Q3 . . . . .                                        | 37        |
| 5.2      | Quanlitative evaluation . . . . .                   | 37        |
| 5.2.1    | Q3 . . . . .                                        | 37        |
| 5.2.2    | Q4 . . . . .                                        | 37        |
| 5.3      | Weather-time assessment . . . . .                   | 38        |
| 5.3.1    | Parameter characterisation tests . . . . .          | 38        |
| 5.3.2    | Realism tests . . . . .                             | 39        |
| 5.3.3    | Method Validation . . . . .                         | 40        |
| <b>6</b> | <b>Experiments</b>                                  | <b>41</b> |
| 6.1      | Flow-time evaluation . . . . .                      | 41        |
| 6.1.1    | Setup . . . . .                                     | 41        |
| 6.1.2    | Achievement tests . . . . .                         | 42        |
| 6.2      | Weather-time evaluation . . . . .                   | 43        |
| 6.2.1    | Parameter characterisation tests . . . . .          | 43        |
| 6.2.2    | Realism tests . . . . .                             | 43        |

---

|                   |                                                        |           |
|-------------------|--------------------------------------------------------|-----------|
| 6.3               | Use cases . . . . .                                    | 43        |
| 6.3.1             | Europe 2020-06-01 . . . . .                            | 43        |
| 6.3.2             | Australia 2020-01-01 . . . . .                         | 43        |
| 6.3.3             | North America 2020-07-01 . . . . .                     | 43        |
| <b>7</b>          | <b>Discussion</b>                                      | <b>44</b> |
| 7.1               | Mask . . . . .                                         | 44        |
| 7.2               | ... . . . .                                            | 45        |
| <b>8</b>          | <b>Conclusion</b>                                      | <b>46</b> |
| 8.1               | Summary . . . . .                                      | 46        |
| 8.2               | Future Directions . . . . .                            | 46        |
| 8.3               | Conclusive Remarks . . . . .                           | 46        |
| <b>Appendices</b> |                                                        |           |
| <b>A</b>          | <b>Development of the target-realization algorithm</b> | <b>48</b> |
| A.1               | CLOSE-GAP . . . . .                                    | 48        |
| A.2               | OPTIMIZE-SCHEDULE . . . . .                            | 50        |

# List of Figures

|     |                                                                                                                                                                                                                                                            |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Computational graph of one ArchesWeather forecast step. The deterministic model produces the base forecast $\hat{x}_{n+1}^{\text{det}}$ , while the flow model generates the residual through $T$ Euler steps before both components are combined. . . . . | 20 |
| 3.1 | Example of the spatial target mask. The rectangular support determines which grid cells are included, while the latitude-dependent weights account for differences in physical grid-cell area. . . . .                                                     | 23 |

# List of Tables

# Chapter 1

## Introduction

Machine-learning (ML) models have reached state-of-the-art skill in weather forecasting, rivaling traditional physics-based numerical models at a fraction of their computational cost [14, 3].

An increasingly important class of these models is formed by generative probabilistic weather models [20, 6]. Rather than producing a single deterministic forecast, these models are trained to approximate the conditional distribution of future atmospheric states. Sampling from the learned distribution enables the direct generation of forecast ensembles and thereby an explicit representation of forecast uncertainty. This probabilistic formulation also mitigates the smoothing effects often associated with deterministic models and improves probabilistic forecast skill, measured for instance by the Continuous Ranked Probability Score (CRPS) [10, 20].

Beyond their use as probabilistic samplers, generative models can also be actively controlled during generation. This is standard in image generation, where applications such as text-to-image synthesis, editing, and inpainting rely on *guidance* to steer a pretrained generative model toward a desired outcome [7, 12, 19]. Generative weather models, by contrast, are still predominantly used in their unguided form, and their potential for controlled generation remains comparatively unexplored.

A small number of recent studies have begun to investigate different forms of weather control for the generation of extreme-event scenarios, including tropical cyclones [4], droughts [26], and heatwaves [28]. These approaches shift the focus from forecasting a distribution of likely weather trajectories to generating plausible low-probability alternatives under prescribed conditions of interest. We refer to this complementary task as *scenario generation*.

This distinction is particularly relevant for extreme-weather storylines. Extreme events ac-

count for a disproportionate share of weather-related impacts [29], yet their rarity means that both historical observations and finite forecast ensembles provide only a limited sample of possible realizations. Considering plausible low-probability alternatives beyond those represented in historical observations and finite forecast ensembles is therefore essential for assessing extreme-weather risk [25, 23].

We are therefore interested in exploiting the ability of generative weather models to produce coherent weather trajectories while deliberately steering them toward prescribed extreme events. Transferring guidance methods from image generation to this setting is nonetheless not straightforward. First, while image-generation targets can often be expressed directly through a text prompt, class label, or reference image, weather scenarios require the conditioning objective to be explicitly constructed. Second, while image plausibility can often be assessed from visual coherence, weather plausibility is inherently multivariate and spatiotemporal: guided trajectories must remain coherent across space and pressure levels, preserve meaningful relationships between meteorological variables, and evolve realistically over time. These considerations lead to the central research question of this thesis:

*Can we guide generative probabilistic weather models to produce extreme weather trajectories that satisfy a prescribed target while remaining meteorologically plausible?*

We address this question by separating two problems that are often intertwined in guided generation: *what* scenario should be generated, and *how* the generative model should be steered to produce it. We refer to these two stages as *Specify* and *Guide*.

The first stage defines the desired scenario independently of the guidance mechanism. In our setting, this amounts to specifying a target variable, a spatial region, and a target trajectory describing how the corresponding regional quantity should evolve over forecast time. The second stage then determines how the generative dynamics must be modified to realize the prescribed target.

This separation leads to a central distinction from the approaches most closely related to ours. Existing guidance methods typically prescribe the strength of the intervention and observe the resulting change in the generated scenario. We instead specify the desired outcome first and adapt the guidance strength to reach it. To do so, we parameterize guidance through a prescribed profile over the flow and determine its overall magnitude separately for each forecast step.

We investigate this approach using ArchesWeather [6], a medium-range probabilistic weather

model based on flow matching. As an experimental case study, we focus on regional surface-temperature heatwaves and specify the target through a prescribed temperature trajectory over a selected spatial region. The underlying formulation is nevertheless not restricted to temperature: the same approach can in principle be applied to other differentiable functionals of the forecast state. More generally, although extreme weather provides the application considered in this thesis, the proposed formulation can be used to generate alternative weather trajectories toward any suitably specified target.

The main contributions of this thesis are the following.

1. We provide a common formulation of guidance-strength schedules that separates the overall guidance magnitude from its distribution over generative flow time.
2. We formulate the specification of a target weather scenario through a spatial mask and a scalar target trajectory, separating the desired outcome from the guidance mechanism used to realize it.
3. We develop a target-driven gradient-based guidance method that prescribes the relative distribution of guidance over flow time and adapts its overall magnitude to the specified target.
4. We systematically investigate how the placement of guidance over flow time affects the resulting scenarios and use the selected configuration to generate and evaluate extreme-weather storylines.

The remainder of the thesis is organized as follows. Chapter 2 introduces the technical background on flow matching and guidance, reviews existing approaches to controlled weather generation, and describes ArchesWeather. Chapter 3 develops the specification of target weather scenarios, while Chapter 4 presents the guidance method used to realize them. ?? introduces the criteria used to assess the resulting trajectories, and Chapter 6 presents the experimental results, which are discussed in Chapter 7. Finally, Chapter 8 summarizes the main findings and outlines directions for future work.

# Chapter 2

## Background

This chapter introduces the theoretical and methodological background required for the development of our guidance approach.

(TODO: reformulate the starting sentence) We begin with flow matching, the generative framework underlying ArchesWeather, and introduce the probability paths, noisy states, velocity fields, and clean estimates used throughout this work. We then formulate guidance from a posterior-sampling perspective, derive its flow-matching formulation, and discuss different approaches to guidance strength and control. Next, we review existing approaches to the controlled generation of weather scenarios and position our work within this emerging literature. Finally, we introduce ArchesWeather, the probabilistic weather model used in this thesis, and describe the components relevant to our method, including its deterministic forecast, residual flow model, and sampling procedure.

### 2.1 Flow matching

Generative modeling frames the task of generating objects  $x_1 \in \mathbb{R}^d$  as sampling from a data distribution  $p_1$ . Flow matching [16] models  $p_1$  as the endpoint of a continuous probability path

$$(p_t)_{t \in [0,1]},$$

which transports a tractable source distribution  $p_0$ , typically a standard Gaussian, to the data distribution  $p_1$ .

At flow time  $t$ , we denote the current state along this path by  $z_t$ , with

$$z_0 \sim p_0, \quad z_t \sim p_t, \quad z_1 = x_1 \sim p_1.$$

Throughout this work, we refer to  $z_0$  as the *noise*, intermediate states  $z_t$ ,  $t \in (0, 1)$ , as *noisy states*, and the endpoint  $z_1 = x_1$  as the *clean state*.

The evolution of  $z_t$  is governed by a time-dependent velocity field  $u_t$ , defining the ordinary differential equation

$$\frac{d}{dt}z_t = u_t(z_t). \quad (2.1)$$

Starting from  $z_0 \sim p_0$  and integrating this ODE from  $t = 0$  to  $t = 1$  therefore transforms samples from the source distribution into samples from the data distribution.

### 2.1.1 Conditional probability paths

Directly specifying the marginal probability path  $p_t$  and its corresponding velocity field  $u_t$  is generally intractable. Flow matching circumvents this problem by defining a tractable *conditional probability path*

$$p_t(\cdot \mid x_1)$$

for each data sample  $x_1 \sim p_1$ .

A common choice is the Gaussian conditional probability path

$$p_t(\cdot \mid x_1) = \mathcal{N}(\alpha_t x_1, \beta_t^2 I), \quad (2.2)$$

where the schedules  $\alpha_t, \beta_t \in [0, 1]$  interpolate between noise and data according to

$$\alpha_0 = 0, \quad \beta_0 = 1, \quad \alpha_1 = 1, \quad \beta_1 = 0.$$

Consequently,

$$p_0(\cdot \mid x_1) = \mathcal{N}(0, I), \quad (2.3)$$

$$p_1(\cdot \mid x_1) = \delta_{x_1}. \quad (2.4)$$

Sampling from the conditional path is equivalent to interpolating a data sample  $x_1 \sim p_1$  with

independent Gaussian noise  $\varepsilon \sim \mathcal{N}(0, I)$ :

$$z_t = \alpha_t x_1 + \beta_t \varepsilon \sim p_t(\cdot \mid x_1). \quad (2.5)$$

Marginalizing over the data distribution gives the corresponding marginal probability path

$$p_t(z_t) = \int p_t(z_t \mid x_1) p_1(x_1) dx_1. \quad (2.6)$$

### 2.1.2 Conditional and marginal velocity fields

Each conditional probability path  $p_t(\cdot \mid x_1)$  is generated by a corresponding conditional velocity field  $u_t(z_t \mid x_1)$ .

For the Gaussian path in Equation (2.2), differentiating the interpolation in Equation (2.5) gives

$$u_t(z_t \mid x_1) = \dot{\alpha}_t x_1 + \dot{\beta}_t \varepsilon. \quad (2.7)$$

Substituting

$$\varepsilon = \frac{z_t - \alpha_t x_1}{\beta_t}$$

yields

$$u_t(z_t \mid x_1) = \frac{\dot{\beta}_t}{\beta_t} z_t + \left( \dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) x_1. \quad (2.8)$$

The marginal velocity field generating  $p_t$  can then be expressed as the conditional velocity averaged over all clean samples  $x_1$  compatible with the current noisy state  $z_t$ :

$$u_t(z_t) = \mathbb{E}_{x_1 \sim p_t(\cdot \mid z_t)} [u_t(z_t \mid x_1)]. \quad (2.9)$$

While this marginal velocity is generally not available in closed form, the conditional velocity in Equation (2.8) is tractable during training, since both  $x_1$  and the sampled noise  $\varepsilon$  are known.

### 2.1.3 Flow-matching objective

Flow matching exploits the fact that a neural network can be trained on the tractable conditional velocity field while learning the corresponding marginal velocity field [16]. Writing

$u_\theta(z_t, t)$  for the learned velocity field, the conditional flow-matching objective is

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}[0,1], \\ x_1 \sim p_1, \\ z_t \sim p_t(\cdot | x_1)}} [\|u_\theta(z_t, t) - u_t(z_t | x_1)\|_2^2]. \quad (2.10)$$

This conditional objective has the same gradient with respect to the model parameters as the corresponding, but intractable, marginal flow-matching objective [16].

After training,  $u_\theta(z_t, t)$  approximates the marginal velocity field  $u_t(z_t)$ . Generation then consists of sampling

$$z_0 \sim p_0$$

and numerically integrating

$$\frac{d}{dt} z_t = u_\theta(z_t, t) \quad (2.11)$$

from  $t = 0$  to  $t = 1$ , such that the resulting endpoint  $z_1$  approximately follows  $p_1$ .

### 2.1.4 Linear probability path

A particularly simple choice is the linear, or optimal-transport, probability path

$$z_t = tx_1 + (1 - t)\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad (2.12)$$

corresponding to

$$\alpha_t = t, \quad \beta_t = 1 - t.$$

Its conditional velocity reduces to the constant displacement

$$u_t(z_t | x_1) = x_1 - \varepsilon. \quad (2.13)$$

Training therefore amounts to least-squares regression onto the displacement between the source noise and the clean sample.

Under the conditional probability path used to construct the marginal flow, an intermediate noisy state  $z_t$  is generally compatible with multiple clean samples  $x_1$ . Their conditional distribution is

$$p_t(x_1 | z_t).$$

This should be distinguished from the deterministic coupling induced by the learned flow ODE.

Once a state  $z_t$  is fixed during sampling, integrating the velocity field from  $t$  to 1 determines a unique endpoint. The distribution  $p_t(x_1 \mid z_t)$  instead refers to the coupling induced by the conditional probability path in Equation (2.2).

In practice, a generative model can provide a tractable clean estimate

$$\hat{x}_t := \hat{x}_\theta(z_t, t), \quad (2.14)$$

from the current noisy state. This point estimate should therefore not be confused with the deterministic endpoint obtained by continuing the learned flow from  $z_t$ . Rather, it provides a representative clean state associated with the conditional distribution  $p_t(x_1 \mid z_t)$ . This distinction becomes important when introducing guidance in the following section.

## 2.2 Guidance

Without additional guidance, a generative model samples from its base distribution  $p_1(x_1)$ . In many applications, however, we are interested in generating samples that satisfy an additional desired condition  $y$ . From a posterior-sampling perspective, guidance aims to modify the sampling process toward the corresponding conditional distribution

$$p_1(x_1 \mid y).$$

We first formulate guidance from this posterior-sampling perspective and translate the resulting score modification to the flow-matching velocity field. We then introduce a general guidance-strength schedule and review different strategies for choosing its magnitude over the generative trajectory. Finally, we discuss a complementary control perspective, in which guided generation is formulated as an optimization problem over variables that influence the complete generative trajectory.

### 2.2.1 Guidance as posterior sampling

To introduce guidance during the generative trajectory, we consider the distribution of noisy states conditioned on  $y$  at an arbitrary flow time  $t$ . By Bayes' rule,

$$p_t(z_t \mid y) \propto p_t(z_t) p_t(y \mid z_t). \quad (2.15)$$

Taking the gradient with respect to the noisy state gives the conditional score

$$\nabla_{z_t} \log p_t(z_t | y) = \nabla_{z_t} \log p_t(z_t) + \nabla_{z_t} \log p_t(y | z_t). \quad (2.16)$$

The first term corresponds to the unguided generative process, while the second introduces the desired condition  $y$ . Guidance can therefore be interpreted as modifying the original generation process through the likelihood gradient

$$\nabla_{z_t} \log p_t(y | z_t).$$

In general, however, the intermediate likelihood  $p_t(y | z_t)$  is not directly available. We therefore marginalize over the clean sample  $x_1$  and assume that the condition  $y$  depends on the generative process only through  $x_1$ :

$$p_t(y | z_t) = \int p_t(y | x_1, z_t) p_t(x_1 | z_t) dx_1 \quad (2.17)$$

$$= \int p(y | x_1) p_t(x_1 | z_t) dx_1, \quad y \perp z_t | x_1 \quad (2.18)$$

$$= \mathbb{E}_{x_1 \sim p_t(x_1 | z_t)} [p(y | x_1)] \quad (2.19)$$

$$\approx \int p(y | x_1) \delta_{\hat{x}_t}(x_1) dx_1, \quad p_t(x_1 | z_t) \approx \delta_{\hat{x}_t}(x_1) \quad (2.20)$$

$$= p(y | \hat{x}_t). \quad (2.21)$$

Here,  $\hat{x}_t = \hat{x}_\theta(z_t, t)$  denotes the clean estimate obtained from the current noisy state. The approximation replaces the full conditional distribution  $p_t(x_1 | z_t)$  by a point mass at  $\hat{x}_t$ , yielding

$$p_t(y | z_t) \approx p(y | \hat{x}_t). \quad (2.22)$$

This clean-estimate approximation also underlies Diffusion Posterior Sampling [5].

The likelihood  $p(y | x_1)$  can in principle be modeled in different ways. A convenient choice for gradient-based guidance is to define it through a differentiable loss  $\mathcal{L}$ :

$$p(y | x_1) \propto \exp(-\mathcal{L}(x_1, y)). \quad (2.23)$$

It follows that

$$\nabla_{z_t} \log p(y | \hat{x}_t) = -\nabla_{z_t} \mathcal{L}(\hat{x}_t, y), \quad (2.24)$$

where the gradient is propagated through the clean estimate  $\hat{x}_t = \hat{x}_\theta(z_t, t)$ .

### 2.2.2 From scores to flow-matching velocities

TODO: cite the MIT notes for the translation formula

The previous derivation expresses guidance in terms of scores. To apply it to flow matching, we require the corresponding modification of the velocity field.

For the Gaussian probability path introduced in Equation (2.2), the marginal velocity field and score are related by

$$u_t(z_t) = a_t \nabla_{z_t} \log p_t(z_t) + b_t z_t, \quad (2.25)$$

where

$$a_t := \beta_t^2 \left( \frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t} \right), \quad (2.26)$$

$$b_t := \frac{\dot{\alpha}_t}{\alpha_t}. \quad (2.27)$$

Applying the same relation to the conditional distribution gives

$$u_t(z_t | y) = a_t \nabla_{z_t} \log p_t(z_t | y) + b_t z_t. \quad (2.28)$$

Substituting the conditional-score decomposition yields

$$u_t(z_t | y) = a_t [\nabla_{z_t} \log p_t(z_t) + \nabla_{z_t} \log p_t(y | z_t)] + b_t z_t \quad (2.29)$$

$$= u_t(z_t) + a_t \nabla_{z_t} \log p_t(y | z_t). \quad (2.30)$$

Using the clean-estimate approximation derived above, we obtain

$$u_t(z_t | y) \approx u_t(z_t) + a_t \nabla_{z_t} \log p(y | \hat{x}_t). \quad (2.31)$$

Under the loss-based likelihood in Equation (2.23), this can equivalently be written as

$$u_t(z_t | y) \approx u_t(z_t) - a_t \nabla_{z_t} \mathcal{L}(\hat{x}_t, y). \quad (2.32)$$

Thus, the posterior-sampling formulation provides a principled local guidance direction: evaluate a loss on the clean estimate  $\hat{x}_t$  and differentiate it with respect to the current noisy state  $z_t$ . The corresponding magnitude is determined by  $a_t$  under the posterior-sampling interpretation, but can more generally be treated as an algorithmic choice, as discussed next.

### 2.2.3 Guidance-strength schedules

The coefficient  $a_t$  in Equation (2.32) is determined by the underlying probability path. Using this coefficient preserves the posterior-sampling interpretation, up to the clean-estimate approximation introduced above.

In practice, guidance methods often rescale this contribution to obtain stronger or weaker control over the generated samples. We therefore replace  $a_t$  by a general guidance-strength schedule

$$\lambda_t := w \gamma_t \nu_t, \quad (2.33)$$

where

- $w \geq 0$  controls the overall guidance strength;
- $\gamma_t \geq 0$  controls how guidance varies along the flow;
- $\nu_t > 0$  is an optional algorithm-specific normalization factor.

We then define the *guided velocity field* as

$$\tilde{u}_t(z_t | y) := u_t(z_t) + \lambda_t \nabla_{z_t} \log p(y | \hat{x}_t) \quad (2.34)$$

$$= u_t(z_t) - \lambda_t \nabla_{z_t} \mathcal{L}(\hat{x}_t, y). \quad (2.35)$$

We refer to  $u_t(z_t)$  as the *unguided velocity field*.

Setting

$$\lambda_t = a_t$$

recovers the approximate conditional velocity field in Equation (2.31). For a general choice of  $\lambda_t$ , however, the resulting dynamics no longer necessarily correspond to sampling from  $p_1(x_1 | y)$ . Instead, the guidance strength becomes an algorithmic parameter controlling the trade-off between the original generative dynamics and the imposed condition.

The choice of  $\lambda_t$  varies substantially across applications. The simplest approach is to use a constant guidance strength throughout sampling,

$$\lambda_t = w, \quad (2.36)$$

where the scalar  $w > 0$  controls the overall strength of guidance and is treated as a tunable hyperparameter. Fixed guidance scales are widely used in classifier-free guidance; for instance, Esser et al. [8] evaluate rectified-flow models using several values of  $w$ . A similar strategy is used for weather guidance by Ueyama, Kawamoto, and Kera [26], who tune a single

guidance scale throughout the GenCast diffusion process, omitting guidance only at the final noise level. Loss-Guided Diffusion [24] similarly tunes an overall task-level guidance strength, while additionally introducing noise-dependent normalization to control the magnitude of the resulting gradient.

Other approaches prescribe the temporal variation of guidance using the noise schedule of the pretrained generative model. In the original classifier-guidance formulation [7], and subsequently in Universal Guidance [1], the guidance contribution is scaled according to

$$\lambda_t = w\sqrt{1 - \bar{\alpha}_t^{\text{diff}}}, \quad (2.37)$$

where  $\bar{\alpha}_t^{\text{diff}}$  denotes the cumulative signal coefficient of the pretrained diffusion process. Consequently,  $\sqrt{1 - \bar{\alpha}_t^{\text{diff}}}$  is the standard deviation of the noise present at diffusion step  $t$ . The overall gain  $w$  is again tuned for the particular guidance objective, while the temporal shape of the schedule is inherited directly from the pretrained model’s noise schedule rather than chosen independently.

Finally, the guidance strength can be adapted dynamically using quantities observed during sampling. Climate in a Bottle [4], for example, normalizes its classifier-guidance contribution relative to the magnitude of the unguided generative update. Schematically, in the notation used here, this corresponds to

$$\lambda_t \propto w \frac{\|u_t(z_t)\|_2}{\|\nabla_{z_t} \mathcal{L}(\hat{x}_t, y)\|_2}, \quad (2.38)$$

such that the guidance vector remains a fixed fraction of the unguided generative update. From an optimization perspective, Shen et al. [22] similarly interpret guidance strength as a step size and propose a Polyak-style adaptive rule. Their update scales the guidance gradient using both its own norm and the magnitude of the unguided model prediction, adapting the effective step size at each sampling step rather than prescribing it beforehand.

These examples illustrate that, even within the same local guidance construction, the guidance schedule is not uniquely determined in practice. Existing approaches range from tuning a constant global gain  $w$ , to combining  $w$  with a temporal schedule inherited from the pretrained model, to adapting the effective guidance magnitude dynamically during sampling.

### 2.2.4 Guidance as control

A complementary perspective formulates guided generation as a control problem over the generative trajectory. Rather than constructing a local approximation of a conditional score,

the objective is to modify variables that influence the trajectory such that its endpoint satisfies a desired condition.

For consistency, we express the following methods using the notation introduced in this chapter rather than that of the original papers. We denote the generative state by  $z_t$ , the frozen pretrained velocity field by  $u_t(z_t)$ , and the terminal guidance objective by  $\mathcal{L}(z_1, y)$ .

FlowGrad [17] introduces an additive control  $g_t$  into the pretrained dynamics,

$$\frac{d}{dt}z_t = u_t(z_t) + g_t, \quad (2.39)$$

and optimizes the complete control trajectory according to

$$\min_{\{g_t\}} \mathcal{L}(z_1, y) + \kappa \int_0^1 \|g_t\|_2^2 dt, \quad (2.40)$$

where  $g_t$  determines how the pretrained dynamics are modified at flow time  $t$ , and  $\kappa > 0$  penalizes large deviations from the original flow. Since the terminal state  $z_1$  depends on all preceding controls, their gradients are obtained by backpropagating the terminal objective through the ODE trajectory.

D-Flow [2] instead leaves the velocity field unchanged and optimizes only the source noise:

$$\min_{z_0} \mathcal{L}(z_1(z_0), y), \quad \frac{d}{dt}z_t = u_t(z_t). \quad (2.41)$$

Here,  $z_1(z_0)$  emphasizes that the generated endpoint is determined by the initial noise through integration of the frozen flow. The terminal gradient is therefore propagated through the complete generative trajectory back to  $z_0$ .

OC-Flow [27] places such approaches within a general optimal-control framework. Using the same notation, its Euclidean formulation can be viewed as optimizing a control trajectory

$$\min_{\{g_t\}} \mathcal{L}(z_1, y) + \int_0^1 \mathcal{C}(g_t) dt, \quad \frac{d}{dt}z_t = u_t(z_t) + g_t, \quad (2.42)$$

where  $\mathcal{C}$  is a running cost that penalizes the applied control. OC-Flow derives the control updates using optimal-control theory and shows that existing backpropagation-through-ODE approaches can be interpreted within this framework.

The posterior-sampling and control perspectives therefore provide two complementary views of guided generation. Posterior-style methods construct local corrections to the generative dynamics from a conditioning objective, while control-based methods formulate guidance as

an optimization problem over variables that influence the complete generative trajectory.

These perspectives provide the conceptual basis for the weather-specific approaches to controlled generation reviewed next.

## 2.3 Guided generation of weather scenarios

Having introduced the posterior-sampling and control perspectives on guidance, we now turn to their application in weather generation. Guided generation in weather and climate models remains comparatively recent, but existing work already spans several settings, from controlling individual atmospheric states to modifying complete weather trajectories. The approaches differ primarily in where control enters the generation process and how the desired event is specified.

For multi-step weather trajectories, we use  $n = 0, \dots, N$  to index weather time, while  $t$  continues to denote the internal diffusion or flow time of the generative process. Sampling-time conditioning has also been explored beyond extreme-event generation, for example by incorporating external forecast information during diffusion sampling [13]. Here, we focus specifically on methods aimed at controlling rare or extreme weather outcomes.

**State-level guidance.** At the most local level, guidance can be used to control the properties of a single generated atmospheric state. A representative example is Climate in a Bottle (cBottle) [4], which uses classifier guidance for the on-demand generation of extreme atmospheric states. Unlike autoregressive weather models, cBottle generates independent multivariate states conditioned on slowly varying quantities such as sea-surface temperature and solar position.

To generate tropical cyclones at user-specified locations, the authors train a classifier on historical cyclone observations and use its gradient during diffusion sampling. Expressed schematically in the notation introduced above, the guided dynamics take the form

$$\tilde{u}_t(z_t \mid y) = u_t(z_t) - \lambda_t \nabla_{z_t} \mathcal{L}_{\text{TC}}(z_t, y), \quad (2.43)$$

where  $y$  specifies the desired tropical-cyclone locations and  $\mathcal{L}_{\text{TC}}$  denotes the classifier loss. The guidance strength is adaptively normalized relative to the unguided generative update, as discussed in Section 2.2.3. The resulting samples contain cyclones at the requested locations while retaining correlated structure across atmospheric variables [4].

Since cBottle generates atmospheric states independently, the method controls the occurrence and location of an event in a single state rather than its temporal evolution.

**Forecast guidance.** Guidance has also been applied directly within probabilistic weather forecasting. Ueyama, Kawamoto, and Kera [26] use gradient-based guidance during GenCast diffusion sampling to reduce precipitation over a prescribed target region. In contrast to approaches that perturb an atmospheric state before forecasting, the intervention is applied within the generative sampling process itself.

Using the guidance notation introduced above, the mechanism can be represented schematically as

$$\tilde{u}_t(z_t | y) = u_t(z_t) - \lambda \nabla_{z_t} \mathcal{L}_{\text{precip}}(\hat{x}_t, y), \quad (2.44)$$

where  $\mathcal{L}_{\text{precip}}$  measures the precipitation-reduction objective on the current clean estimate  $\hat{x}_t$ . The gradient is propagated through the clean estimate to the noisy state  $z_t$  and used to modify the generative update.

The guidance scale  $\lambda$  is fixed during sampling and controls the strength of the intervention. Increasing it produces stronger precipitation reduction, but also larger deviations from the original forecast. Accordingly, the authors evaluate not only whether the intervention succeeds, but also its effects on the surrounding atmospheric state through variable-wise and vertical perturbation structures, deviations of the generative trajectory, and cross-model transferability [26].

**Trajectory-level control.** Control objectives can also be defined over complete weather trajectories. A particularly relevant example is Whittaker and Di Luca [28], who construct extreme heatwave storylines by exploiting the differentiability of NeuralGCM. Starting from a historical atmospheric state  $x_0$ , they optimize a small perturbation  $\delta x_0$  to the initial condition whose subsequent evolution produces a more extreme heatwave:

$$\delta x_0^* = \arg \min_{\delta x_0} [\mathcal{L}_{\text{HW}}(F_{0:N}(x_0 + \delta x_0)) + \kappa \|\delta x_0\|_2^2], \quad (2.45)$$

where  $F_{0:N}$  denotes the differentiable NeuralGCM rollout,  $\mathcal{L}_{\text{HW}}$  penalizes trajectories that fail to attain the desired heatwave intensity, and  $\kappa > 0$  controls the penalty on the magnitude of the initial perturbation.

Gradients are propagated through the complete model trajectory back to the initial atmo-

spheric state. Applied to the 2021 Pacific Northwest heatwave, the optimization produces trajectories more extreme than those found in a conventional ensemble while retaining characteristic large-scale structures of the event [28].

This construction differs from the generative-guidance approaches above. NeuralGCM is used as a differentiable dynamical model, and control is achieved by optimizing the initial atmospheric condition rather than by intervening within a generative sampling process. It therefore provides an example of trajectory-level control through initial-condition optimization.

Other approaches control trajectories without differentiating an event loss through the forecast model. He and Guo [9] use Intensity–Duration–Frequency profiles as likelihood constraints during sequential generation, preferentially retaining trajectories compatible with the desired extreme profile. Related rare-event methods instead clone or prune trajectories according to their likelihood of developing into the target event [15].

A further alternative is to incorporate the desired condition during training rather than impose it on a pretrained generator at inference time. Loer, Schillinger, and Sippel [18], for example, learn a conditional generative distribution of temperature fields given atmospheric circulation and global warming level. In the Bayesian terminology introduced above, this corresponds to learning  $p(x | y)$  directly, rather than starting from a pretrained model of  $p(x)$  and introducing the condition through  $p(y | x)$  during sampling.

Taken together, these studies illustrate several routes to controlled weather generation: guiding individual atmospheric states, intervening during probabilistic forecast generation, optimizing initial conditions for complete trajectories, selectively sampling rare trajectories, and learning conditional generators directly.

## 2.4 ArchesWeather

We investigate this problem using ArchesWeather [6], a probabilistic medium-range weather forecasting model that combines a deterministic forecast with a flow-matching model of the forecast residual. Having established the general flow-matching and guidance formulations above, we now describe the components of ArchesWeather required for the development of our method and refer to the original work for architectural and training details.

### 2.4.1 Dataset

ArchesWeather is trained on the ERA5 reanalysis dataset [11], using one of the versions preprocessed by WeatherBench 2 [21]. The data are represented on a regular  $1.5^\circ$  latitude–longitude grid with  $121 \times 240$  spatial points.

Each weather state

$$x_n \in \mathbb{R}^{82 \times 121 \times 240}$$

contains 82 atmospheric fields. Four are surface variables: 2 m temperature, mean sea-level pressure, and the two components ( $u$  and  $v$ ) of the 10 m wind. The remaining 78 fields correspond to six upper-air variables

$$\{\text{geopotential, temperature, specific humidity, } u, v, \text{ vertical velocity}\}$$

evaluated at the 13 pressure levels

$$\{50, 100, 150, 200, 250, 300, 400, 500, 600, 700, 850, 925, 1000\} \text{ hPa.}$$

Weather states are available at 00:00, 06:00, 12:00, and 18:00 UTC. Following the WeatherBench 2 evaluation protocol, dates from January 2020 onward are reserved for evaluation.

Both the deterministic and generative components operate in normalized data space. Unless stated otherwise, the variables introduced below therefore refer to their normalized representation.

### 2.4.2 Forecasting setup

ArchesWeather generates forecast trajectories autoregressively. Following the convention introduced above, we index forecast lead time by  $n$ , which we refer to as *weather time*. At each weather step, the model produces a forecast for the next state  $x_{n+1}$ .

Rather than generating  $x_{n+1}$  directly, ArchesWeather decomposes the forecast into a deterministic component and a stochastic residual component. The deterministic model first produces a base forecast

$$\hat{x}_{n+1}^{\text{det}}.$$

The corresponding ground-truth residual is defined as

$$r_{n+1} := x_{n+1} - \hat{x}_{n+1}^{\text{det}}. \tag{2.46}$$

A conditional flow-matching model is then trained to generate this residual. At inference time, a sampled residual  $\hat{r}_{n+1}$  is combined with the deterministic prediction as

$$\hat{x}_{n+1} = \hat{x}_{n+1}^{\text{det}} + \hat{r}_{n+1}. \quad (2.47)$$

Sampling different residuals while keeping the deterministic forecast fixed therefore produces different members of an ensemble forecast.

### 2.4.3 Residual flow model

Generating each stochastic residual requires a complete flow-matching trajectory. ArchesWeather therefore involves two nested notions of time: the weather-time index  $n$ , which advances the autoregressive forecast, and the flow time  $t$ , which describes the generation of a residual within each weather step.

Following the notation introduced in the previous sections, the clean endpoint of the flow is now the forecast residual  $r_{n+1}$ , while  $z_t$  denotes its noisy state at flow time  $t$ .

ArchesWeather parameterizes the Gaussian probability path in terms of a noise level  $s_t$ , which decreases linearly from 1 at the beginning of the flow to 0 at its endpoint. The noisy residual is constructed as

$$z_t = (1 - s_t) r_{n+1} + s_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I). \quad (2.48)$$

This corresponds directly to the Gaussian probability path introduced in Equation (2.2), with

$$\alpha_t = 1 - s_t, \quad \beta_t = s_t. \quad (2.49)$$

The flow therefore starts from Gaussian noise and progressively approaches the clean residual  $r_{n+1}$ .

Unlike the velocity-field parameterization introduced in the generic formulation above, the ArchesWeather residual model is trained to predict the clean residual from the noisy state. We denote this estimate by

$$\hat{r}_t := r_\theta(z_t, t, c_n), \quad (2.50)$$

where

$$c_n = (x_{n-1}, x_n, \hat{x}_{n+1}^{\text{det}})$$

collects the conditioning information provided to the residual model.

The model is trained using the clean-prediction objective

$$\mathcal{L}_{\text{Arches}}(\theta) = \mathbb{E}_{t, r_{n+1}, \varepsilon} [\|r_\theta(z_t, t, c_n) - r_{n+1}\|_2^2]. \quad (2.51)$$

In the notation of the previous sections,  $\hat{r}_t$  thus plays the role of the clean estimate  $\hat{x}_t$ . Since the flow generates a residual rather than the complete weather state, the corresponding clean weather estimate at flow time  $t$  is

$$\hat{x}_t := \hat{x}_{n+1}^{\text{det}} + \hat{r}_t. \quad (2.52)$$

This quantity will later provide the state on which our guidance objectives are evaluated.

### 2.4.4 Sampling

Although the network predicts the clean residual, sampling requires the corresponding velocity field. From Equation (2.48), the noise associated with the clean estimate  $\hat{r}_t$  is

$$\hat{\varepsilon}_t = \frac{z_t - (1 - s_t)\hat{r}_t}{s_t}. \quad (2.53)$$

For the linear path

$$\alpha_t = 1 - s_t, \quad \beta_t = s_t,$$

the velocity is the displacement between the clean residual estimate and the corresponding noise estimate:

$$u_\theta(z_t, t, c_n) = \hat{r}_t - \hat{\varepsilon}_t \quad (2.54)$$

$$= \hat{r}_t - \frac{z_t - (1 - s_t)\hat{r}_t}{s_t} \quad (2.55)$$

$$= \frac{\hat{r}_t - z_t}{s_t}. \quad (2.56)$$

ArchesWeather discretizes the flow into  $T = 25$  sampling steps. In the following,  $t = 0, \dots, T$  denotes the corresponding discrete flow-step index. Starting from Gaussian noise

$$z_0 \sim \mathcal{N}(0, I),$$

the residual is generated through Euler integration,

$$z_{t+1} = z_t + h_t u_\theta(z_t, t, c_n), \quad h_t = s_t - s_{t+1}, \quad t = 0, \dots, T-1. \quad (2.57)$$

Since the noise level decreases from  $s_0 = 1$  to  $s_T = 0$ , the sequence  $\{z_t\}_{t=0}^T$  progressively evolves from noise toward a clean residual.

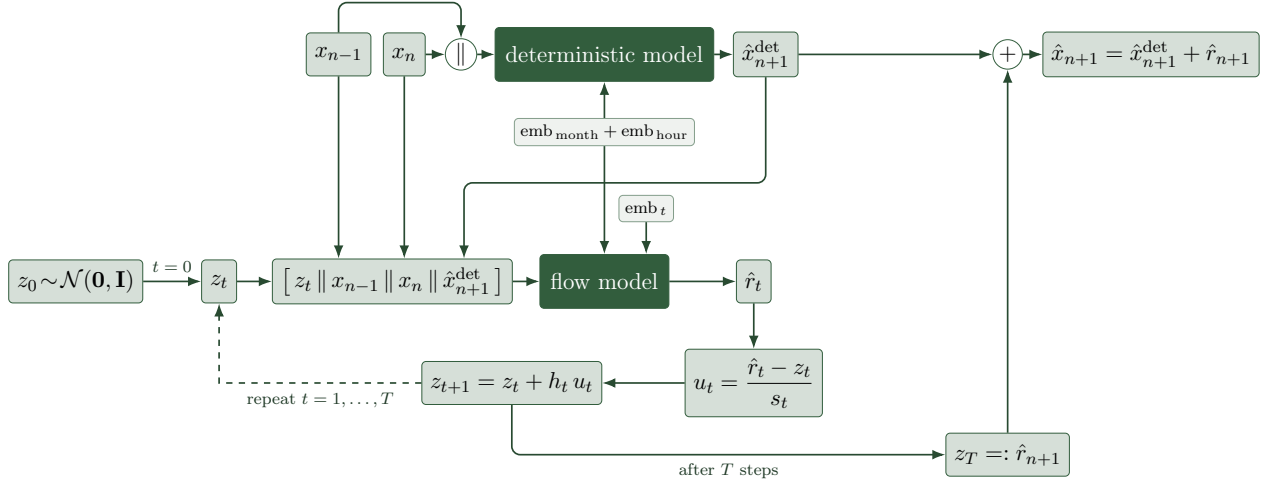
After the final flow step, the endpoint is taken as the sampled residual,

$$\hat{r}_{n+1} := z_T, \quad (2.58)$$

which is added to the deterministic forecast to obtain

$$\hat{x}_{n+1} = \hat{x}_{n+1}^{\text{det}} + \hat{r}_{n+1} = \hat{x}_{n+1}^{\text{det}} + z_T. \quad (2.59)$$

Figure 2.1 summarizes the computations performed within a single weather step. This structure will be central to the implementation and evaluation of our guidance method.



**Figure 2.1** – Computational graph of one ArchesWeather forecast step. The deterministic model produces the base forecast  $\hat{x}_{n+1}^{\text{det}}$ , while the flow model generates the residual through  $T$  Euler steps before both components are combined.

The nested residual-flow structure of ArchesWeather provides the concrete generative process on which our guidance method will operate. Before developing that method in Chapter 4, we first specify the weather trajectories toward which the model should be steered in Chapter 3.

# Chapter 3

## Specifying events of interest

The first stage of our approach addresses *what* weather scenario should be generated. We specify an event through two components: a mask  $\pi$  that selects the atmospheric variable and spatial region of interest, and a target trajectory (TODO: insert y here with correct notation) describing the desired evolution of the corresponding regional quantity over the guided forecast horizon.

Throughout this thesis, we focus on regional surface-temperature heatwaves. Accordingly,  $\pi$  selects surface temperature over a prescribed region, while the target trajectory specifies how its regional average should evolve over the guided weather steps. The formulation is nevertheless not restricted to temperature and can be applied to other variables or (TODO: reformulate) differentiable regional functionals of the forecast state.

(TODO: improve introductory sentence) As generative model, ArchesWeather weather can generate ensemble forecasts by drawing multiple samples. To generate weather scenarios we want to specify the guidance target to retain the properties of the ensemble. Rather than guiding all members toward the same absolute trajectory, we define a common relative displacement and apply it to each member with respect to its matched unguided forecast. This preserves the member-specific structure of the target trajectories instead of imposing an artificial collapse onto a single value.

We first define the target variable and region through the spatial mask  $\pi$ . We then construct the target trajectory over the guided horizon and derive the corresponding member-specific targets to use in the guidance algorithm.

### 3.1 Target variable and region

We jointly specify the target variable and spatial region through a mask  $\pi$  over the weather state. Let

$$x_n \in \mathbb{R}^{V \times I \times J}$$

denote a weather state, where  $V$  indexes atmospheric fields, while  $I$  and  $J$  index latitude and longitude. We denote the corresponding spatial coordinates by

$$(\varphi_i, \psi_j), \quad i = 1, \dots, I, \quad j = 1, \dots, J.$$

Let  $v^*$  denote the target variable, and let

$$k_{ij} \in \{0, 1\}$$

indicate whether spatial grid cell  $(i, j)$  lies inside the selected target region. In this thesis, the spatial support is defined through a rectangular bounding box on the latitude–longitude grid. The mask is non-zero only for the selected variable and within the selected region. To obtain an area-weighted regional average, each included grid cell is weighted according to the physical surface area that it represents. For a regular latitude–longitude grid, the area contribution of latitude row  $i$  is proportional to

$$a_i = \int_{\beta_i^-}^{\beta_i^+} \cos \varphi \, d\varphi, \quad (3.1)$$

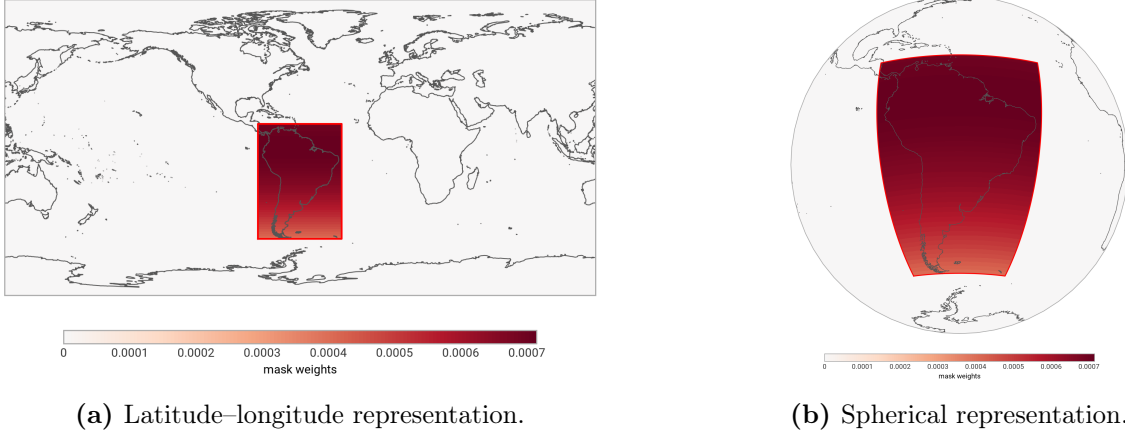
where  $[\beta_i^-, \beta_i^+]$  denotes the latitude extent of the corresponding grid cell. Since the longitudinal grid spacing is constant, its contribution cancels during normalization.

We therefore define the normalized mask as

$$\pi_{vij} = \frac{\mathbf{1}[v = v^*] k_{ij} a_i}{\sum_{i', j'} k_{i'j'} a_{i'}}, \quad (3.2)$$

such that

$$\sum_{v, i, j} \pi_{vij} = 1.$$



**Figure 3.1** – Example of the spatial target mask. The rectangular support determines which grid cells are included, while the latitude-dependent weights account for differences in physical grid-cell area.

The corresponding masked-average operator is

$$\mathcal{M}_\pi(x_n) := \sum_{v,i,j} \pi_{vij} x_{n,vij}. \quad (3.3)$$

For the experiments considered in this thesis,  $\mathcal{M}_\pi(x_n)$  therefore represents the area-weighted mean surface temperature within the selected target region.

Area weighting prevents the regional average from being biased toward high-latitude grid cells, which represent a smaller physical area on the sphere. This follows the latitude-weighting convention used in WeatherBench 2 [21].

Figure 3.1 illustrates the resulting area-weighted mask for an example target region over South America. Its spatial support is rectangular on the latitude-longitude grid, while the normalized weights vary with latitude according to the physical area represented by each grid cell.

TODO: improve the figure by having a single colorbar on the side and the globe has the same diameter as the side of the rectangle.

Using a binary spatial support makes the target region explicit. Within the selected region, grid cells differ only through their physical area and are otherwise treated uniformly. This avoids introducing an additional spatial preference, as would occur, for example, with a bell-shaped mask whose center is weighted more strongly than its boundaries. Normalizing  $\pi$  to sum to one ensures that  $\mathcal{M}_\pi$  remains an average and is therefore directly comparable across regions of different sizes.

Although  $\pi$  selects one particular variable and region, its spatial support  $k_{ij}$  can be reused to construct analogous regional averages for other atmospheric fields. This becomes useful when evaluating how guidance applied to surface temperature affects other variables and pressure levels over the same region.

## 3.2 Target trajectory

Once the target variable and region have been specified through the mask  $\pi$ , the remaining task is to define the desired evolution of the targeted quantity. Let  $H \leq N$  denote the final guided weather step. We first describe this evolution at the ensemble level through the target trajectory

$$y_{1:H}^* = (y_1^*, \dots, y_H^*),$$

and subsequently translate it into member-specific targets  $y_{1:H,m}^*$ . No target is imposed for  $n > H$ .

TODO: make sure that  $N$  is actually defined in the background, it also doesn't hurt to respecify it here.

### 3.2.1 Guiding an ensemble

At each weather step, ensemble members generally attain different values of the targeted quantity  $\mathcal{M}_\pi(x_n)$ , with their spread typically increasing as forecast uncertainty grows with lead time.

Assigning the same absolute target trajectory to every member would ignore these differences and prescribe zero ensemble spread in  $\mathcal{M}_\pi(x_n)$ . (TODO: check wording) We therefore define the guidance target as desired change relative to each member's own matched unguided trajectory.

Each guided rollout is paired with a matched unguided rollout

$$\{x_{n,m}^{\text{ung}}\}_{n=1}^N, \quad m = 1, \dots, M,$$

using the same initial atmospheric context and matched stochastic realizations. At weather step  $n$ , we prescribe a common relative change  $\rho_n$  across the ensemble and define the target of member  $m$  as

$$y_{n,m}^* := (1 + \rho_n) \mathcal{M}_\pi(x_{n,m}^{\text{ung}}), \quad n = 1, \dots, H. \quad (3.4)$$

The corresponding ensemble-level target is the mean of the member-specific targets,

$$y_n^* := \frac{1}{M} \sum_{m=1}^M y_{n,m}^* = (1 + \rho_n) \frac{1}{M} \sum_{m=1}^M \mathcal{M}_\pi(x_{n,m}^{\text{ung}}). \quad (3.5)$$

Applying the same relative change to all members retains the differences between their matched unguided trajectories. Let  $\text{std}_m[\cdot]$  denote the standard deviation across the ensemble-member index  $m = 1, \dots, M$  at a fixed weather step  $n$ . Then, for  $1 + \rho_n > 0$ , the spread of the member-specific targets satisfies

$$\text{std}_m[y_{n,m}^*] = (1 + \rho_n) \text{std}_m[\mathcal{M}_\pi(x_{n,m}^{\text{ung}})]. \quad (3.6)$$

The target ensemble therefore inherits the lead-time-dependent spread of the matched unguided ensemble, scaled by the prescribed relative change, rather than collapsing all members onto a common trajectory. Whether the guided ensemble reflects the spread implied by these member-specific targets ultimately depends on the dynamics of the guidance algorithm.

### 3.2.2 Prescribing the relative change

It remains to specify the relative change  $\rho_n$ . In principle, the shape and magnitude of  $\rho_{1:H}$  could be informed by historical events, climatological statistics, or other application-specific definitions of the scenario of interest. Historical events could, for example, provide a reference for how the requested relative intensity evolves over time.

(TODO: state this also in the intro for consistency) The focus of this thesis, however, is to evaluate the behaviour of the guidance method under controlled settings. We therefore prescribe  $\rho_n$  directly and use a simple linear increase over the guided horizon,

$$\rho_n := \frac{n}{H} \rho_H, \quad n = 1, \dots, H, \quad (3.7)$$

where  $\rho_H$  denotes the maximum requested relative change and  $H$  the final guided weather step.

This construction gradually increases the requested deviation rather than imposing its full magnitude from the first forecast step. Because the relative change is applied to each member's

matched unguided value through Equation (3.4), the target trajectory continues to inherit the individual temporal evolution of that member while progressively moving away from its unguided forecast. At step  $H$ , the prescribed relative change reaches  $\rho_H$  and guidance is subsequently stopped.

For the surface-temperature experiments considered here,  $\rho_H$  is therefore a scale-dependent experimental intensity parameter rather than a climatological measure of heatwave extremeness. TODO: specify this how we will use it in our experiments (we will define multiple levels of intensity and assess results under the different levels)

### 3.3 Pipeline overview

In practical terms, the elements specified throughout this chapter are changed together as follows. Starting from an initial atmospheric context  $(x_{-1}, x_0)$ , we first define the mask  $\pi$ , which specifies the target variable and spatial region. For each ensemble member  $m = 1, \dots, M$ , we then generate a matched unguided rollout

$$\{x_{n,m}^{\text{ung}}\}_{n=1}^N,$$

which serves as the reference trajectory for constructing the target.

A relative intensity trajectory  $\rho_{1:H}$  is prescribed over the guided horizon and applied to each matched unguided member according to Equation (3.4). This yields the set of member-specific target trajectories

$$\{y_{1:H,m}^{\star}\}_{m=1}^M.$$

The output of the specification stage is therefore

$$\left(\pi, \{y_{1:H,m}^{\star}\}_{m=1}^M\right), \tag{3.8}$$

where  $\pi$  determines which atmospheric quantity is targeted, while the member-specific trajectories determine its desired evolution during the guided forecast horizon. For  $n > H$ , no target is imposed and the forecast continues without guidance until the end of the rollout at  $N$ .

TODO: include here the rollout trajectories planning figure.

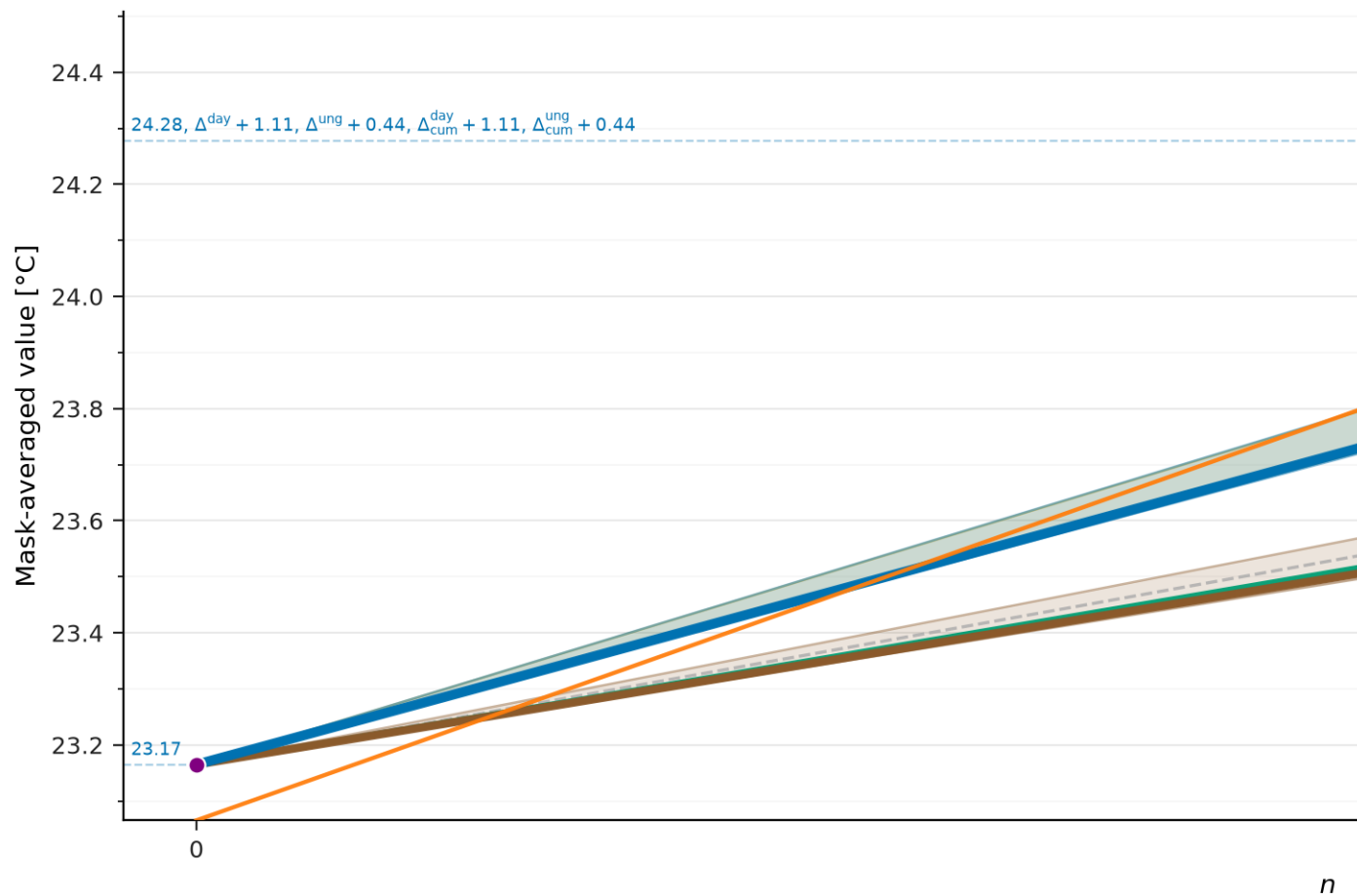
These objects define *what* the model should generate independently of *how* the generative

process is modified to realize them. The next chapter addresses this complementary guidance problem.

TODO: somewhere around here we should show the graph that this creates.

### rollout trajectories

2m\_temperature | mask-averaged



# Chapter 4

## Guidance design

The previous chapter specified the scenario to be generated through a mask  $\pi$  and a set of member-specific target trajectories. We now address the second component of the framework: how the generative forecast should be steered toward these prescribed targets.

Because guidance is applied independently to each ensemble member, we suppress the ensemble-member index throughout this chapter. Accordingly,  $y_n^*$  denotes the target associated with the member currently being generated.

(TODO: this can probably be specified better connecting to something previous instead of restating again) We formulate the problem as *target-driven guidance*. Rather than choosing a guidance strength beforehand and observing the resulting change in the forecast, the desired outcome is specified first. Guidance strength then becomes a means of realizing this target. At each weather step, we use the target discrepancy evaluated on the intermediate clean-state estimates to construct a local guidance direction, while the magnitude of the intervention is adapted separately to close the remaining target gap at the end of the generative flow.

Several strategies can be considered for this target-realization problem. One possibility is to adapt the intervention locally such that the target gap follows a prescribed evolution during the flow. A more general alternative is to optimize the guidance strengths at all flow steps jointly. In this work, we instead prescribe the relative distribution of guidance over flow time and adapt only a single scalar controlling its overall magnitude. Preliminary investigations of the former approaches are discussed in Appendix A; the scalar-gain formulation retained here provides the basis for the experiments in this thesis.

TODO: here we can state that it becomes clear from the experiments why this formulation doesn't demand for any optimization of the schedule under the DPS formulation.

## 4.1 Target-driven guidance

ArchesWeather generates a forecast trajectory autoregressively. Given the atmospheric context available at weather step  $n$ , the model generates the next state  $x_{n+1}$ . Guidance is applied within the residual flow used to generate this state.

Although the guidance problem is solved separately at each weather step, its effects propagate through weather time. Once a guided state  $x_{n+1}^{\text{gui}}$  has been generated, it becomes part of the conditioning context used to generate the following state. Consequently, guidance applied at one weather step can influence all subsequent states of the guided forecast trajectory.

### 4.1.1 Guidance objective

At weather step  $n$ , the specification procedure introduced in Chapter 3 provides the target value  $y_{n+1}^*$  for the state being generated. During the residual flow, ArchesWeather provides a clean-state estimate  $\hat{x}_{n+1,t}$  at each flow step  $t$ . We define its discrepancy from the prescribed target as

$$\xi_{n+1,t} := \mathcal{M}_\pi(\hat{x}_{n+1,t}) - y_{n+1}^*, \quad (4.1)$$

where  $\mathcal{M}_\pi$  denotes the masked functional defined in Chapter 3. We refer to  $\xi_{n+1,t}$  as the *target gap* at flow step  $t$ .

The corresponding guidance objective is the squared target gap,

$$\mathcal{L}_{n+1,t} := \xi_{n+1,t}^2 = [\mathcal{M}_\pi(\hat{x}_{n+1,t}) - y_{n+1}^*]^2. \quad (4.2)$$

Evaluating the objective on the clean-state estimate follows the local clean-estimate guidance formulation introduced in ???. It makes the conditioning objective available throughout the generative flow and therefore provides a guidance direction at every intermediate state.

An alternative would be to define the objective only on the final generated state and differentiate it backwards through the complete flow. Such a formulation is possible and corresponds to the control perspective discussed in Section 2.2.4. The distinction is therefore not whether the final state can be differentiated through, but where the intervention is defined. Here, we retain the local posterior-style construction: the clean estimate determines the guidance direction at each flow step, while the resulting terminal target gap is used separately to determine how strongly this direction must be applied.

For the experiments in this thesis,  $\mathcal{M}_\pi$  is the area-weighted regional mean surface temperature introduced in Chapter 3. The guidance construction itself is more general, however, and can be applied to other differentiable functionals of the forecast state.

### 4.1.2 Gradient-based flow guidance

We index the discrete residual flow such that  $z_{n+1,0}$  denotes the initial noise and  $t = 1, \dots, T$  indexes the successive flow updates,

$$z_{n+1,0} \xrightarrow{1} z_{n+1,1} \xrightarrow{2} \dots \xrightarrow{T} z_{n+1,T}. \quad (4.3)$$

At flow step  $t$ , the model evaluates the velocity field and clean-state estimate from the current noisy state  $z_{n+1,t-1}$ . The local guidance direction is obtained by differentiating the loss in Equation (4.2) with respect to this state,

$$g_{n+1,t} := \nabla_{z_{n+1,t-1}} \mathcal{L}_{n+1,t}. \quad (4.4)$$

For the squared target-gap objective, this can be written as

$$g_{n+1,t} = 2\xi_{n+1,t} \nabla_{z_{n+1,t-1}} \mathcal{M}_\pi(\hat{x}_{n+1,t}). \quad (4.5)$$

Although the loss is defined through the masked quantity  $\mathcal{M}_\pi(\hat{x}_{n+1,t})$ , the gradient is propagated through the mapping from the noisy residual state to the complete clean-state estimate. The resulting direction therefore describes how the current noisy state can be modified, through the learned generative model, to change the targeted forecast quantity. It is not restricted to modifying only the entries selected by the mask.

Following the loss-based guidance formulation introduced in the background chapter, the velocity field is modified according to

$$\tilde{u}_{n+1,t} = u_{n+1,t} - \lambda_{n,t} g_{n+1,t}, \quad (4.6)$$

where  $\lambda_{n,t} \geq 0$  controls the guidance magnitude at weather step  $n$  and flow step  $t$ . The resulting Euler update is

$$z_{n+1,t} = z_{n+1,t-1} + h_t \tilde{u}_{n+1,t}, \quad t = 1, \dots, T. \quad (4.7)$$

The local guidance direction is therefore fully determined by the specified target and the current generative state. What remains to be determined is the sequence of guidance magnitudes  $\lambda_{n,t}$  required to realize the prescribed target. We address this design problem next.

## 4.2 Guidance-strength design

The background formulation in Section 2.2.3 decomposes the guidance strength into an overall magnitude, a flow-time profile, and an optional normalization factor. Applied independently at each weather step, this gives

$$\lambda_{n,t} = w_n \gamma_t \nu_{n,t}, \quad (4.8)$$

where  $w_n$  controls the overall guidance magnitude used to generate  $x_{n+1}$ ,  $\gamma_t$  determines its relative variation over flow time, and  $\nu_{n,t}$  represents an optional additional normalization.

In our method, we apply the raw guidance gradient without an additional normalization and therefore set

$$\nu_{n,t} = 1. \quad (4.9)$$

The guidance strength consequently reduces to

$$\lambda_{n,t} = w_n \gamma_t. \quad (4.10)$$

We prescribe  $\gamma_t$  as a non-negative flow-time profile and normalize it such that

$$\max_{t=1,\dots,T-1} \gamma_t = 1. \quad (4.11)$$

Only the relative variation of  $\gamma_t$  over flow time is therefore encoded by the profile, while its overall magnitude is carried entirely by  $w_n$ . Conceptually, the two quantities address different aspects of the guidance problem:  $\gamma_t$  determines *where* along the generative flow guidance is applied, whereas  $w_n$  determines *how strongly* it is applied at the current weather step. The profile families considered in this thesis are introduced together with the experimental design. Substituting Equation (4.10) into Equation (4.6) gives the update used throughout the main experiments,

$$\tilde{u}_{n+1,t} = u_{n+1,t} - w_n \gamma_t g_{n+1,t}. \quad (4.12)$$

### 4.2.1 Alternative target-realization strategies

The decomposition above leaves several possibilities for determining the guidance strengths required to realize the prescribed target. During the development of the method, we considered three increasingly constrained formulations.

A first approach adapts the guidance magnitude locally at every flow step such that the remaining target gap follows a prescribed trajectory toward zero. This directly controls the evolution of the gap during generation, but requires specifying how quickly the target should be approached within the internal flow.

A second possibility treats the individual guidance strengths  $\lambda_{n,1}, \dots, \lambda_{n,T-1}$  as independent optimization variables and determines them jointly from the final target discrepancy. This provides substantially more flexibility, but replaces the scalar guidance-strength choice by a higher-dimensional optimization problem.

The formulation retained in this thesis instead fixes their relative structure through  $\gamma_t$  and adapts only the scalar gain  $w_n$ . Preliminary experiments showed that this restricted formulation was sufficient to realize the prescribed targets while reducing the target-realization problem to a single dimension. The additional flexibility of the former approaches was therefore not required for the experiments considered here. Their formulations and their role in the development of the final method are documented in Appendix A.

### 4.2.2 Final flow step

Guidance is applied during the first  $T - 1$  flow updates but omitted from the final update. This convention follows directly from the clean-prediction parameterization of ArchesWeather. At the final flow step, the current residual state is  $z_{n+1,T-1}$  and the remaining noise level is  $s_T$ . From Equation (2.56), the corresponding velocity is

$$u_{n+1,T} = \frac{\hat{r}_{n+1,T} - z_{n+1,T-1}}{s_T}. \quad (4.13)$$

Because the final Euler step integrates the remaining noise level completely, its step size is  $h_T = s_T$ . Without guidance,

$$z_{n+1,T} = z_{n+1,T-1} + s_T u_{n+1,T} \quad (4.14)$$

$$= z_{n+1,T-1} + \hat{r}_{n+1,T} - z_{n+1,T-1} \quad (4.15)$$

$$= \hat{r}_{n+1,T}. \quad (4.16)$$

The final update therefore maps the noisy residual state directly onto the current clean-residual prediction. A guidance correction applied at this step would perturb the resulting residual without any subsequent model evaluation in which the modified state could be processed. We consequently set

$$\gamma_T = \lambda_{n,T} = 0, \quad (4.17)$$

such that  $t = T - 1$  is the final guided flow step. Omitting guidance at the final noise level is also consistent with the precipitation-guidance procedure of Ueyama, Kawamoto, and Kera [26].

### 4.3 Target realization by gain adaptation

Once the flow-time profile  $\gamma_t$  has been fixed, the only remaining guidance parameter at weather step  $n$  is the scalar gain  $w_n$ . Rather than choosing this gain beforehand, we determine it from the target that was specified for the state  $x_{n+1}$ .

For a fixed atmospheric context, flow-time profile, and initial noise realization  $z_{n+1,0}$ , running the complete guided residual flow defines the generated state as a function of the gain,

$$x_{n+1}^{\text{gui}}(w_n). \quad (4.18)$$

We define the corresponding terminal target gap by

$$\xi_{n+1}(w_n) := \mathcal{M}_\pi \left( x_{n+1}^{\text{gui}}(w_n) \right) - y_{n+1}^*. \quad (4.19)$$

This quantity differs from the intermediate gap  $\xi_{n+1,t}$  in Equation (4.1). The latter is evaluated on a clean estimate during the flow and determines the local guidance direction, whereas  $\xi_{n+1}(w_n)$  is evaluated on the final generated state and measures whether the prescribed target was actually realized.

The target-realization problem is therefore the scalar root-finding problem

$$\xi_{n+1}(w_n^*) = 0. \quad (4.20)$$

This formulation directly reflects the target-driven nature of the method: the desired endpoint is fixed first, and the role of  $w_n$  is solely to determine the guidance magnitude required to reach it.

### 4.3.1 Secant search

Since evaluating  $\xi_{n+1}(w_n)$  requires running the complete guided residual flow, we seek its root without computing derivatives with respect to the gain. We use the secant method, which constructs a local linear approximation from two previously evaluated gains.

Given  $w_n^{(j-1)}$  and  $w_n^{(j)}$ , the next candidate is

$$w_n^{(j+1)} = w_n^{(j)} - \xi_{n+1}(w_n^{(j)}) \frac{w_n^{(j)} - w_n^{(j-1)}}{\xi_{n+1}(w_n^{(j)}) - \xi_{n+1}(w_n^{(j-1)})}. \quad (4.21)$$

The first evaluation uses  $w_n = 0$ , corresponding to generation of the current weather step without flow guidance, and a second evaluation uses a positive initial gain. For later weather steps, the  $w_n = 0$  evaluation should not be confused with the matched unguided trajectory used to define the target in Chapter 3. Previous guided states already form part of the autoregressive conditioning context, so removing guidance from the current residual flow does not in general recover the independently generated unguided forecast.

All evaluations performed by the secant search use the same initial noise realization  $z_{n+1,0}$ . Consequently, the mapping

$$w_n \longmapsto \xi_{n+1}(w_n)$$

changes only through the guidance magnitude and not through stochastic differences between repeated flow samples. Importantly, the local guidance gradients  $g_{n+1,t}$  are nevertheless recomputed during every evaluation, since changing  $w_n$  changes the sequence of noisy states and therefore the clean-state estimates encountered along the flow.

The secant iteration is restricted to non-negative gains. If the target is reached within the prescribed tolerance, the corresponding gain is retained. If no root is obtained within the numerical safeguards or evaluation budget, we instead select the evaluated value that leaves

the smallest absolute terminal gap,

$$w_n^\star := \arg \min_{w \in \mathcal{W}_n} |\xi_{n+1}(w)|, \quad (4.22)$$

where  $\mathcal{W}_n$  denotes the set of gains evaluated during the search. The concrete initialization, convergence tolerance, step safeguards, and maximum number of evaluations are specified together with the experimental setup.

This procedure is repeated independently at every guided weather step. The resulting sequence of gains

$$w_0^\star, \dots, w_{H-1}^\star$$

therefore adapts the overall guidance magnitude to the member-specific target at each point along the prescribed trajectory. For weather states beyond the guided horizon,  $n + 1 > H$ , no target is imposed and the autoregressive forecast continues without guidance.

# Chapter 5

## Evaluation

We evaluate the developed algorithm against the two requirements we have set for it:

- Quantitative evaluation.
  - Q1: Is it able to achieve the target we have set for it, across different experimental settings?
  - Q2: Which guidance schedule yields the most desirable results?
  - Q3: What results do the different experimental settings achieve, and what is their impact?
  - Q4: Are the results realistic in terms of spatial distribution, levels distribution, and co-variation, and time progression?

We organize the evaluation evaluation as follows. We answer the first two questions quantitatively and proceed with a more qualitative approach for the latter two concerns. While address the third question with both quantitative metrics but with a qualitative approach, aimed only at characterize different settings. The last question would call for a quantitative approach, however, this would require both expert knowledge and a thorough set of tests involving historical data, which is out of scope for this thesis. We thus address each one of the subquestions, but only assessing the visual characteristics.

## 5.1 Quantitative evaluation

### 5.1.1 Q2

### 5.1.2 Q3

## 5.2 Quantitative evaluation

### 5.2.1 Q3

### 5.2.2 Q4

Here we assess the algorithm under different guidance schedules, with the purpose of selecting the most adequate parameters to perform the weather-time evaluation.

Before asking whether the method produces realistic results, we first have to establish that it works: that it reaches the target, does so reliably across settings, and does so in a desirable manner (stability and deviation concerns).

We test the achievement requirement and well-behavedness of the trajectory for different guidance algorithms through the following tests:

#### **T1 – Reliability**

The gap to the target is driven to zero. We report the mean and standard deviation of the final gap.

#### **T2 – Stability**

The trajectory is free of oscillatory behaviour. We measure this simply with respect to the pushback measured in the average trajectory (gui vs. ung steps) and whether the algorithm overshoots.

#### **T3 – Closeness**

We check that the guided trajectory does not deviate excessively from the unguided one. We report the total amount of guidance needed, and the pathlength in PCA latent space, and the cosine similarity between the steps, compared to the gui-ung run.

TODO: set a bad initialization on purpose

## 5.3 Weather-time assessment

In weather-time we first characterise the parameters of the experimental setting—to describe their properties and to select settings for the larger experiments—and then validate the realism of the resulting trajectories. Throughout, we pay special attention to the diversity of the produced results as a secondary property of interest, alongside realism.

### 5.3.1 Parameter characterisation tests

In the following we provide a set of qualitative tests to better understand the role of the main parameters of the guidance at weather time. We test parameters in isolation, using the remaining parameters as controls. We proceed in three stages, examining in turn the region of interest, the variable of interest, and the intervention profile. For each we show different charts and metrics to expose what happens across these settings.

— maybe here is where the baseline would shine.

The reason for these tests is mostly to enhance our intuition about the role of each parameter, and provide a guideline on to select parameters for the large use cases we aim to produce with this work.

#### Region of interest

Here we assess variability of results by changing size, shift (from center), and coast-land covering of region of the interest.

#### Variable of interest

For simplicity we focus on heatwaves, and therefore on surface temperature. To probe the informativeness of the gradient, in this test we push the most correlated variables in an attempt to reproduce the pattern obtained by pushing temperature directly.

#### Intervention profile

Here we assess the degradation of the target patterns across intensity profiles. The intuition is that at some point things should brake down in some trivial way.

### 5.3.2 Realism tests

Once the guidance method and its parameters are selected, we assess the realism of the generated trajectories. Realism is central to this work, because we deliberately guide the generative model, possibly outside its training distribution. The tests themselves, however, are not specific to this work: they concern the general problem of evaluating the coherence of data-driven models, namely that the generated states should be consistent with the ground-truth dataset in both the guided and unguided settings.

We assess coherence to the ground-truth dataset along four dimensions:

#### **T4 (Temporal).**

Transitions of a variable across weather steps should be smooth.

#### **T5 (Spatial).**

Patterns over the region of interest should be spatially coherent.

#### **T6 (Vertical).**

Patterns should be coherent across pressure levels. We test this using a level rank coherence metric.

#### **T7 (Cross-variable).**

Patterns should be coherent with the learned weather transition function across variables. However, this is difficult to test quantitatively, we thus only make a qualitative assessment. (TODO: use the expert knowledge here)

We benchmark these properties against the ground-truth data as anchor (reference set), and against the unguided forecast as baseline, comparing unguided, guided-minus-unguided, and guided states. For T4–T6 we compute difference maps along the axis of interest and report the min, mean, max, and standard deviation across maps. For T7 we compute the cross-variable distributions over the weather steps of the ground-truth data, and report the deviation of the unguided, guided-minus-unguided, and guided states from their average.

For all four tests we average over the experiments and report the mean and standard deviation across runs. This indicates whether the generated extremes are faithful to the data distribution. A subtle but important point: for this to be meaningful, the base forecast must have missed the potential heat spots. The extreme is not only about the average—which the forecast most likely covers faithfully—but about the extreme spots, so both the mean and the max must be tested.

Chapter 6 sets out the concrete settings for each stage of this framework and reports the results, which Chapter 7 then discusses.

### 5.3.3 Method Validation

This approach can be used to guide any state towards an extreme over the specified mask.

To validate that the method is doing something sensible we can retrieve real heatwaves from the test dataset (everything that is not included in the train set), and observe what the model is doing.

Another way would be to guide it towards the ground truth and observe the RMSE of other variables compared to the ung.

Validate method against: gt, opposite direction, same direction. Make sure you apply same amount of guidance needed when defining y star.

# Chapter 6

## Experiments

TODO: before starting with the quantitative evaluation, start just by showing some guidance effect maps and absolute guided states with the rollout trajectories plot.

This chapter carries out the evaluation framework of Chapter 5. Each section follows the same pattern: we first state the concrete settings, then report the results and the decision they lead to. Tests are referenced by the identifiers introduced in Chapter 5 (T1–T7). —actually reference the latex identifiers like for equations

### 6.1 Flow-time evaluation

Note-taking:

- it seems like the gradient is most informative when we only evaluate it once (spike@ instead of spread), and we do it at the  $t=2$ . With informativeness we mean in terms of the push charts.

#### 6.1.1 Setup

We first investigate how the placement of guidance within the generative flow affects target realization. To isolate the effect of flow time, we keep the target region and intensity fixed while varying the atmospheric starting state and stochastic realization. We use the following setup:

- one fixed target mask over Europe;
- three different atmospheric starting states;

- $N = 1$  weather step and  $M = 3$  ensemble members;
- one target intensity of  $+0.25\%$ ;
- three single-step guidance profiles: `spike@1`, `spike@3`, `spike@6`, `spike@12`, `spike@24`.

Each spike profile applies guidance at only one flow step, allowing early, intermediate, and late interventions to be compared directly. The guidance gain  $w_n$  is optimized independently for each experiment using the target-realization algorithm described in Section 4.3. The resulting design comprises  $3 \times 3 \times 5 = 45$  guided experiments.

Given the results of this first test, we now test something related. The question is then not only during which phase of the guidance it's most appropriate to guide, but whether to spread the guidance over multiple steps would be beneficial. We test this by defining `a_t` to be interpolated between spike 1-3, 3-6, ... .

Of course optimizing this would be most appropriate. What we are trying to do here is to observe empirically whether some properties emerge before deferring future research to the development of an appropriate algorithm, which could use this information for initialization.

### 6.1.2 Achievement tests

The difficulty of tuning the first and second algorithm make them less suitable than the third one. Regarding algorithm 1, having a lower computational complexity is useless in practice if we then have to perform multiple costly runs. Regarding algorithm 2, having a desirability requirement encoded in the loss itself only complicate things. Tuning the optimization of this algorithms outweighs the benefits, and induces a much higher cost in practical terms to the algorithm.

At later stages, the gradient becomes more similar to the mask (is more diluted, less sparse), which makes the guided pattern at the end of the flow be more spread.

Using a `spike@1` approach is essentially equivalent to shifting the starting point of the flow trajectory (especially for the region over the mask) and let particle flow across the learned ODE.

What we observe is that the model doesn't necessarily pushes back against us. In contrary, when the method is tuned well, the model will do most of the pushing itself (look at this in percentage).

## 6.2 Weather-time evaluation

Throughout the weather-time experiments we vary the start state while fixing  $N = 2$  and  $M = 3$ : this controls for the effects of  $N = 1$  and for seed-specific effects, while keeping computational and memory requirements as low as possible.

Note that we will use the same experiments for the parameter characterization tests and for the realism tests.

— the realism tests could simply be used in each parameter subsection

### 6.2.1 Parameter characterisation tests

Region of interest

Variable of interest

Intervention profile

### 6.2.2 Realism tests

## 6.3 Use cases

For the final use cases we fix the parameters selected above and increase  $N$  and  $M$ , producing the experiments we had in mind when setting the goals of the project.

### 6.3.1 Europe 2020-06-01

### 6.3.2 Australia 2020-01-01

### 6.3.3 North America 2020-07-01

# Chapter 7

## Discussion

### 7.1 Mask

We experimented with a gaussian mask, but realized in our experiments that the best thing to do is equal weighting inside the region. The correct way to define the region of interest is to choose a greater mask size instead of using some weighting scheme fading out of an epicenter. The mask size should be chosen to cover the entire area of interest, ensuring that all relevant data is included in the analysis. A larger mask size can help to reduce noise and improve the accuracy of the results, while a smaller mask size may exclude important information. It is important to carefully consider the appropriate mask size for each specific application to achieve optimal results. Either way we (most probably) loose the global coherence for the guided weather states, so we might as well define directly the region of interest with a mask. Additionally, this enables us to actually make statements about "the region of interest", having equal weights everywhere. Introducing different weights might bias the generation in a way that is not desired.

We will see that including or not a specific area will change the guided patterns substantially. This however is consistent with the notion of "region of interest". If we consider a greater area for the changes, the changes will be applied respectively.

— note here: it may also be that the pushed area is coherent with the rest, and that the next deterministic prediction influences other regions too. It is really important to understand how to evaluate what we are doing here. Since we are aiming at a specific region of interest, and we are guiding mainly locally over the region of interest, losing the global coherence of the weather states. We shall thus include this in our evaluation by comparing inside region, outside region, and full spatial domain.

## 7.2 ...

The algorithm doesn't produce diverse results, because the gradients are really stable. This has to do with the fact that we have learned one and only one flow model (transition function). If we had multiple residual diffusion model, we would expect the possible effects to be more diverse. However, the second interpretation might be that the gradient provides with the information about the most likely direction of change (which is true under the learned data distribution).

This happens because the guidance vector is a scaled version of the flow model's gradient wrt to the noisy state. The gradient is stable across the full trajectory, because the only variable inputs are the noise itself and the timestep of the flow. The stochasticity of the noise is not enough to produce diverse results. The direction of the gradient is always the same. If we say "let the masked average go up by  $\phi$  percent", the only thing that changes is the magnitude of the gradient, but not its direction, which is specified by the Jacobian.

This is in contrast to guidance settings.

Shift between atmosphere and stratosphere.

Put here all observations

# Chapter 8

## Conclusion

### 8.1 Summary

### 8.2 Future Directions

### 8.3 Conclusive Remarks

# Appendices

# Appendix A

## Development of the target-realization algorithm

Before arriving at the target-realization algorithm presented in Section 4.3, we explored two more flexible strategies for determining the guidance magnitude: CLOSE-GAP and OPTIMIZE-SCHEDULE. These preliminary approaches motivated the final choice to prescribe the relative flow-time profile and optimize only a single overall guidance gain.

### A.1 CLOSE-GAP

The first approach exploits the fact that the desired target is known before generation by prescribing how the remaining target gap should evolve throughout the flow. Let

$$\bar{\xi}_{n,t} := \gamma_t^{\text{gap}} \xi_{n,0}, \quad (\text{A.1})$$

denote the desired remaining gap at flow step  $t$ , where  $\gamma_t^{\text{gap}}$  specifies the fraction of the initial gap that should remain.

At each flow step, the method computes the local displacement of the noisy state required to move the current gap  $\xi_{n,t}$  towards  $\bar{\xi}_{n,t}$ . Using a first-order Taylor approximation,

$$\xi_{n,t}(z_{n,t} + \Delta z_{n,t}) \approx \xi_{n,t}(z_{n,t}) + (\nabla_{z_{n,t}} \xi_{n,t})^\top \Delta z_{n,t}. \quad (\text{A.2})$$

Requiring the corrected gap to equal the prescribed value gives

$$(\nabla_{z_{n,t}} \xi_{n,t})^\top \Delta z_{n,t} = \bar{\xi}_{n,t} - \xi_{n,t}. \quad (\text{A.3})$$

Among the displacements satisfying this local constraint, the smallest in Euclidean norm is obtained by moving along the gradient of the gap,

$$\Delta z_{n,t}^* = \frac{\bar{\xi}_{n,t} - \xi_{n,t}}{\|\nabla_{z_{n,t}} \xi_{n,t}\|_2^2} \nabla_{z_{n,t}} \xi_{n,t}. \quad (\text{A.4})$$

Using

$$g_{n,t} = \nabla_{z_{n,t}} \xi_{n,t}^2 = 2\xi_{n,t} \nabla_{z_{n,t}} \xi_{n,t}, \quad (\text{A.5})$$

the desired displacement can equivalently be written as

$$\Delta z_{n,t}^* = -\frac{2\xi_{n,t} (\xi_{n,t} - \bar{\xi}_{n,t})}{\|g_{n,t}\|_2^2} g_{n,t}. \quad (\text{A.6})$$

The guidance contribution to the Euler update is

$$\Delta z_{n,t}^{\text{gui}} = -h_t \lambda_{n,t} g_{n,t}. \quad (\text{A.7})$$

Requiring this contribution to realize the desired local displacement yields

$$\lambda_{n,t}^{\text{CG}} = \frac{2\xi_{n,t} (\xi_{n,t} - \bar{\xi}_{n,t})}{h_t \|g_{n,t}\|_2^2}. \quad (\text{A.8})$$

The attraction of this formulation is its efficiency: the guidance magnitude is determined online and only a single flow is required. However, the update is greedy. It computes the intervention required to satisfy the prescribed gap under a local approximation, without accounting for the subsequent displacement induced by the learned velocity field. The resulting trajectory can therefore overshoot the prescribed gap and require compensating corrections at later flow steps.

A slower gap-closing profile can reduce this behaviour, but then introduces a schedule parameter controlling how aggressively the gap is closed. Choosing this parameter requires repeated runs, reducing the practical advantage of the otherwise single-pass method. For this reason, we next considered optimizing the flow-time intervention schedule directly.

## A.2 OPTIMIZE-SCHEDULE

Rather than prescribing the evolution of the target gap, OPTIMIZE-SCHEDULE treats the intervention magnitudes across the complete flow as optimization variables.

We parameterize the intervention through its contribution to the Euler update,

$$k_{n,t} := h_t \lambda_{n,t}, \quad (\text{A.9})$$

such that

$$z_{n,t+1} = z_{n,t} + h_t u_{n,t} - k_{n,t} g_{n,t}. \quad (\text{A.10})$$

The quantity  $k_{n,t} g_{n,t}$  is the state displacement induced directly by guidance at flow step  $t$ . We optimize the complete intervention trajectory

$$\mathbf{k}_n = (k_{n,0}, \dots, k_{n,T-2}) \quad (\text{A.11})$$

using the objective

$$J_n(\mathbf{k}_n) = \xi_n^2 + \kappa \sum_{t=0}^{T-2} \|k_{n,t} g_{n,t}\|_2^2. \quad (\text{A.12})$$

The first term measures target realization, while the second penalizes the total squared magnitude of the guidance-induced state displacements. This allows the optimization to prefer schedules that approach the prescribed target while limiting the amount of intervention applied along the flow.

In principle, this formulation removes the need to prescribe a gap-closing trajectory and allows the temporal distribution of guidance to adapt freely. In practice, however, the optimization is strongly coupled. Changing  $k_{n,t}$  modifies the noisy trajectory, which changes the clean-state estimates and therefore also the guidance gradients encountered at subsequent flow steps. Consequently, the optimal intervention at one flow step depends on the interventions applied throughout the trajectory.

In preliminary experiments, this high-dimensional optimization was strongly sensitive to its initialization and could converge rapidly to different local solutions. The additional flexibility therefore came at the cost of a substantially more difficult optimization problem.

These observations motivated the lower-dimensional formulation adopted in the main method.

Instead of determining an independent intervention magnitude at every flow step, we prescribe a normalized profile  $\gamma_t$  and restrict the guidance schedule to

$$\lambda_{n,t} = w_n \gamma_t. \tag{A.13}$$

Target realization then reduces to optimizing the single scalar  $w_n$  at each weather step, as described in Section 4.3.

# Bibliography

- [1] Arpit Bansal et al. “Universal Guidance for Diffusion Models”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. 2023, pp. 843–852.
- [2] Heli Ben-Hamu et al. “D-Flow: Differentiating through Flows for Controlled Generation”. In: *arXiv preprint arXiv:2402.14017* (2024). DOI: 10.48550/arXiv.2402.14017. URL: <https://arxiv.org/abs/2402.14017>.
- [3] Kaifeng Bi et al. “Accurate Medium-Range Global Weather Forecasting with 3D Neural Networks”. In: *Nature* 619 (2023), pp. 533–538. DOI: 10.1038/s41586-023-06185-3.
- [4] Noah D. Brenowitz et al. “Climate in a Bottle: Towards a Generative Foundation Model for the Kilometer-Scale Global Atmosphere”. In: *arXiv preprint arXiv:2505.06474* (2025). DOI: 10.48550/arXiv.2505.06474. URL: <https://arxiv.org/abs/2505.06474>.
- [5] Hyungjin Chung et al. “Diffusion Posterior Sampling for General Noisy Inverse Problems”. In: *arXiv preprint arXiv:2209.14687* (2023). DOI: 10.48550/arXiv.2209.14687. URL: <https://arxiv.org/abs/2209.14687>.
- [6] Guillaume Couairon et al. “ArchesWeather & ArchesWeatherGen: A Deterministic and Generative Model for Efficient ML Weather Forecasting”. In: *arXiv preprint arXiv:2412.12971* (2024). DOI: 10.48550/arXiv.2412.12971.
- [7] Prafulla Dhariwal and Alexander Nichol. “Diffusion Models Beat GANs on Image Synthesis”. In: *Advances in Neural Information Processing Systems*. Vol. 34. 2021, pp. 8780–8794.
- [8] Patrick Esser et al. “Scaling Rectified Flow Transformers for High-Resolution Image Synthesis”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 12606–12633.

- [9] Kanxuan He and Hongshan Guo. “Probabilistic Sequence Generation Guided by Intensity–Duration Extreme Profiles”. In: *SPIGM Workshop at ICML*. 2026. URL: <https://openreview.net/forum?id=n8bQJs1dip>.
- [10] Hans Hersbach. “Decomposition of the Continuous Ranked Probability Score for Ensemble Prediction Systems”. In: *Weather and Forecasting* 15.5 (2000), pp. 559–570. DOI: 10.1175/1520-0434(2000)015<0559:D0TCRP>2.0.CO;2.
- [11] Hans Hersbach et al. “The ERA5 global reanalysis”. In: *Quarterly Journal of the Royal Meteorological Society* 146.730 (2020), pp. 1999–2049. DOI: 10.1002/qj.3803.
- [12] Jonathan Ho and Tim Salimans. “Classifier-Free Diffusion Guidance”. In: *arXiv preprint arXiv:2207.12598* (2022). DOI: 10.48550/arXiv.2207.12598.
- [13] Zhanxiang Hua et al. “Weather Prediction with Diffusion Guided by Realistic Forecast Processes”. In: *arXiv preprint arXiv:2402.06666* (2024). DOI: 10.48550/arXiv.2402.06666. URL: <https://arxiv.org/abs/2402.06666>.
- [14] Remi Lam et al. “Learning Skillful Medium-Range Global Weather Forecasting”. In: *Science* 382.6677 (2023), pp. 1416–1421. DOI: 10.1126/science.adi2336.
- [15] Amaury Lancelin et al. “AI-Boosted Rare Event Sampling to Characterize Extreme Weather”. In: *arXiv preprint arXiv:2510.27066* (2025). DOI: 10.48550/arXiv.2510.27066. URL: <https://arxiv.org/abs/2510.27066>.
- [16] Yaron Lipman et al. “Flow Matching for Generative Modeling”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [17] Xingchao Liu et al. “FlowGrad: Controlling the Output of Generative ODEs With Gradients”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, pp. 24335–24344.
- [18] Frieder Loer, Maybritt Schillinger, and Sebastian Sippel. “Probabilistic Storyline Attribution Using Machine Learning”. In: *arXiv preprint arXiv:2606.02550* (2026). DOI: 10.48550/arXiv.2606.02550. URL: <https://arxiv.org/abs/2606.02550>.
- [19] Alexander Quinn Nichol et al. “GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models”. In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. Proceedings of Machine Learning Research. PMLR, 2022, pp. 16784–16804.
- [20] Ilan Price et al. “Probabilistic Weather Forecasting with Machine Learning”. In: *Nature* 637.8044 (2025), pp. 84–90. DOI: 10.1038/s41586-024-08252-9.

- [21] Stephan Rasp et al. “WeatherBench 2: A Benchmark for the Next Generation of Data-Driven Global Weather Models”. In: *Journal of Advances in Modeling Earth Systems* 16.6 (2024), e2023MS004019. DOI: 10.1029/2023MS004019.
- [22] Yifei Shen et al. “Understanding and Improving Training-Free Loss-Based Diffusion Guidance”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024. DOI: 10.52202/079017-3460.
- [23] Theodore G. Shepherd et al. “Storylines: An Alternative Approach to Representing Uncertainty in Physical Aspects of Climate Change”. In: *Climatic Change* 151.3–4 (2018), pp. 555–571. DOI: 10.1007/s10584-018-2317-9.
- [24] Jiaming Song et al. “Loss-Guided Diffusion Models for Plug-and-Play Controllable Generation”. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 32483–32498.
- [25] Vikki Thompson et al. “High Risk of Unprecedented UK Rainfall in the Current Climate”. In: *Nature Communications* 8 (2017), p. 107. DOI: 10.1038/s41467-017-00275-3.
- [26] Ayumu Ueyama, Kazuhiko Kawamoto, and Hiroshi Kera. “Guided Diffusion Sampling for Precipitation Forecast Interventions”. In: *arXiv preprint arXiv:2605.14317* (2026). DOI: 10.48550/arXiv.2605.14317. URL: <https://arxiv.org/abs/2605.14317>.
- [27] Luran Wang et al. “Training Free Guided Flow-Matching with Optimal Control”. In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: <https://openreview.net/forum?id=61ss5RA1MM>.
- [28] Tim Whittaker and Alejandro Di Luca. “Pushing the Limits of Extreme Weather: Constructing Extreme Heatwave Storylines with Differentiable Climate Models”. In: *arXiv preprint arXiv:2506.10660* (2025). DOI: 10.48550/arXiv.2506.10660.
- [29] World Meteorological Organization. *Atlas of Mortality and Economic Losses from Weather, Climate and Water-Related Hazards (1970–2021)*. World Meteorological Organization, 2023.

## Eidesstattliche Erklärung

Der/Die Verfasser/in erklärt an Eides statt, dass er/sie die vorliegende Arbeit selbständig, ohne fremde Hilfe und ohne Benutzung anderer als die angegebenen Hilfsmittel angefertigt hat. Die aus fremden Quellen (einschliesslich elektronischer Quellen) direkt oder indirekt übernommenen Gedanken sind ausnahmslos als solche kenntlich gemacht. Die Arbeit ist in gleicher oder ähnlicher Form oder auszugsweise im Rahmen einer anderen Prüfung noch nicht vorgelegt worden.

.....  
Ort, Datum

.....  
Unterschrift des/der Verfassers/in